Design note

The five questions in the brief

Question	Our answer	Lives in
What must a learner provide for an attempt to be meaningful?	Java code against a skeleton, plus assumptions and a one-line rationale for the pattern used and the pattern rejected. Code shows structure; the text shows intent, which rules cannot see.	`Submission`, `Rationale`
What makes feedback useful when several designs are valid?	Every finding cites evidence from the learner's own code, says why it matters, and proposes a fix. Judgment is scored against decision points that list several accepted approaches, not one reference answer. Each finding is labelled FACT or JUDGMENT.	`Finding`, `DecisionPoint`
What is deterministic and what is LLM?	Deterministic: anything countable from structure (size, cycles, literals, switch on type, missing capability). LLM: cohesion, pattern fit, trade-offs, and how well the change request was absorbed. An LLM finding that cites a class or method not in the code is dropped.	`Rule`, `LlmDesignReviewEvaluator`, `FindingValidator`
How would another format or approach fit later?	A new format is a new `SubmissionParser` registered under its format key that emits the same `DesignModel`. A new approach is a new `Evaluator` that declares its stage. Neither change touches the other side.	`ParserRegistry`, `Evaluator`
What happens when evaluation is slow or fails?	FACT findings are saved and shown before the LLM runs. The LLM stage runs in the background with a timeout and bounded retries. If it still fails the attempt ends PARTIAL with a retry button. If nothing parses it ends FAILED with the parse errors.	`AttemptLifecycle`, `EvaluationOrchestrator`

Class design

Ports-and-adapters monolith. Immutable domain records in the middle, two extension interfaces (`SubmissionParser`, `Evaluator`), and thin adapters at the edge.

```
web ──> services ──> ports/domain <── adapters
                     model, problem,    JavaCodeParser, Gemini/Demo/Fake LlmClient,
                     evaluation,        JpaAttemptRepository, YamlProblemRepository,
                     attempt            ExecutorEvaluationScheduler, MermaidClassDiagramRenderer
```

Dependency rules, enforced by `ArchitectureTest` (it scans imports):

- `model` imports nothing from the project. `problem` imports `model`. `evaluation` imports `model` and `problem` only.
- `web` talks to services only, never to a repository, an evaluator or the LLM.
- Domain records contain no Spring types.

Key types:

- `DesignModel` is the one format-independent picture of a design. Every evaluator reads only this. It includes `ClassNode` (with `signatureHash` and `bodyHash`), `MethodNode` and `MethodSignals` (facts from a method body so rules never touch source text), and `Relationship` (EXTENDS, IMPLEMENTS, HAS_A, USES).
- `Attempt` changes status only through `apply(event)`, which asks `AttemptLifecycle`.
- `EvaluatorRun` records what each evaluator did, so a failure is visible and never silent.

Patterns used, and why

Pattern	Where	Why this one
Strategy	`Rule`, `Evaluator`	A new check is a new class. The orchestrator never changes.
Registry	`ParserRegistry`	Selects the parser by format and fails fast on an unknown one.
Adapter	`GeminiLlmClient`, `DemoLlmClient`, `FakeLlmClient` behind `LlmClient`	Provider types stay out of the domain; the fake plugs into the same port for tests.
Repository	`AttemptRepository`, `ProblemRepository`	The domain does not know about H2 or YAML.
Transition table	`AttemptLifecycle`	The six states differ only in which transitions are legal, not in behaviour.
Immutable value objects	`Submission`, `DesignModel`, `Finding`	Safe to share between the immediate stage, the background stage and the history diff.

Rejected on purpose: GoF State classes for the lifecycle (six near-empty classes); Observer or an event bus (one process, the orchestrator calls directly); Factory and Builder beyond Spring injection (nothing needs polymorphic creation); Decorator. Visitor appears only inside `JavaCodeParser`, because JavaParser provides it. Over-engineering with unneeded patterns is a named interview mistake, so we left them out.

Evaluation

Stage one (milliseconds, synchronous): parse, then seven rules produce FACT findings, saved before anything else happens. Stage two (background): one LLM review produces JUDGMENT findings.

The seven rules: `GOD_CLASS`, `TYPE_SWITCH`, `HARDCODED_VALUE`, `CONCRETE_DEPENDENCY`, `DEPENDENCY_CYCLE` (Tarjan strongly connected components), `ORPHAN_OR_UNUSED`, `CAPABILITY_OWNERSHIP`. Parser warnings are saved as FACT findings with rule id `PARSE_ISSUE`. Thresholds are starting defaults in `RuleThresholds`.

The LLM sees requirements, decision points, assumptions and rationale, the model as compact text (no method bodies), and the facts already found. Temperature 0, a neutral template, a fixed category list, at most 8 findings.

`FindingValidator` runs on every reply and records how many findings it dropped and why in the `EvaluatorRun` message.

Failure handling

Situation	Result
No class parses from any file	FAILED, parse errors shown
Syntax error outside a method body in one file	That file is skipped with a warning; other files count
Syntax error inside a method body	Structure kept, warning added
Compile errors, missing imports, unknown types	Ignored; we never compile
LLM timeout or transport error	One retry after a back-off, then PARTIAL
LLM returns malformed JSON	One repair retry that includes the parse error, then PARTIAL
LLM cites classes or methods that do not exist	Those findings dropped and counted; attempt still COMPLETED
A rule throws	That rule is skipped and noted; the others still report
Resubmit while evaluating	A new attempt; the earlier one finishes on its own
Retry pressed twice	Second event is an illegal transition, rejected with a clear message
Server restarts mid-evaluation	`StaleEvaluationRecovery` moves it to PARTIAL ("interrupted") so retry works

Each row above has a test.

Deviations from the build spec

These were decisions made while building. They should be reviewed.

1. **Fingerprint has an optional discriminator.** The spec says `ruleId + subject`. Two missing capabilities on the same entry type would then merge in the history diff, so `Finding` carries an optional discriminator (the capability id, or the parse-issue line). For every other finding the fingerprint is exactly `ruleId|subject`.
2. **"Restates a FACT" is narrower than "same subject".** Dropping every LLM finding on a subject that has any rule finding would discard useful judgment. `FindingValidator` drops an LLM finding only when a FACT on the same subject is from a related rule (for example `COHESION` vs `GOD_CLASS`). The mapping is a small table in that class.
3. **Startup recovery is not deadline-based.** At startup nothing can be running in this process, so every attempt still SUBMITTED or EVALUATING is treated as interrupted. A deadline test would leave a just-crashed attempt stuck until the deadline passed.
4. **The service, not the orchestrator, hands the deferred stage to the scheduler.** This avoids a cycle (orchestrator → scheduler → orchestrator). The behaviour is the same.
5. **`ProblemCatalog` service** wraps `ProblemRepository` so the web layer never touches a repository.

6. **Extra endpoint** `POST /attempts/{id}/edit-again` implements the "Edit and resubmit" button for FAILED attempts: a new draft that keeps the learner's code.
7. **Demo mode** (no `GEMINI_API_KEY`) returns a clearly labelled placeholder so the app runs without an account.
8. **The entry-signature exclusion is narrower.** The spec ignores any method that repeats an entry-type method's name and parameter count. Run against the Elevator problem this wrongly rejected `Elevator.step()` (behind `ElevatorSystem.step()`), the natural mover. Now only the entry type's own *public* methods are excluded as owners; a same-named method in another class counts as the delegation it is, and a private helper in the entry type can own a capability. Tested.
9. **Fields carry their declared type.** `FieldNode.type` stays the unwrapped name used for relationships, and a new `declaredType` keeps what the learner wrote (`List<Elevator>`). The AI prompt and the diagram use it. Before this, the model was told only `Elevator` and made two wrong claims about the code (a "single reference" and a "raw TreeSet").
10. **Enum constants, field names and class names count as weaker evidence for a capability.** A design that handles doors through a `DOORS_OPEN` state and no `open()` method was getting a false HIGH "nobody handles doors". `ClassNode` now records enum constants, and when no method matches, `CAPABILITY_OWNERSHIP` reports MEDIUM ("implied, give it a method") if those words appear elsewhere, and HIGH only when there is no trace at all.
11. **Longer LLM time limits than the spec's defaults.** The spec suggests 30 s per call and 90 s overall. A real Gemini review of an Elevator design took 29.6 s in the server log, one second under the limit, so a slightly slower reply would have been cut off and retried. Defaults are now 60 s per call and 150 s overall (`EvaluationProperties`, `lld.llm.timeout-seconds`).
12. **The parser is thread-safe.** `JavaCodeParser` is a singleton used from request threads and the background thread. JavaParser keeps mutable state, so two overlapping parses could report valid code as a syntax error. Each thread now has its own parser and printer; a 400-call, 8-thread test failed before the fix.
13. **Light / Dark / System theme toggle** in the header, remembered in the browser; the diagram is redrawn in the matching colours.
14. **Java level.** The platform builds for Java 17 and was run on 21. The learner's code is parsed at Java 17 as the spec says.

Limits (also in the README)

Structure is read, never run. Java only, top-level types only. The AI sees signatures, not bodies. Thresholds are untuned. `CAPABILITY_OWNERSHIP` matches name fragments. Blast-radius cut-offs are heuristics. The live Gemini call and the in-browser diagram were not exercised in this build.

Not done

Cut as the spec allows: countdown timer, the "disagree" button on findings, the alternative-approaches reveal, and a second (Mermaid) parser to prove the extension point. Not done, and worth doing before trusting the AI review: checking it against 3 to 4 hand-written good and bad solutions per problem with the real model. Hand-written good and bad Parking Lot samples exist and are used by the rule and flow tests, but not for Elevator or Vending Machine.